triskope

see everything together

Production Launch Roadmap

What it will take to move triskope from working prototype to a real SaaS that takes payments, captures leads from agent-branded sites, and grants or denies access to subscribers.

PREPARED FOR
Thomas Bass — Platform Owner

DOCUMENT VERSION
v1.0 • May 12, 2026

HOW TO USE THIS DOCUMENT
Each phase has a checklist (☐). Tick items as they ship. Phases A through F are required for a working paid SaaS. Phase G can be partially deferred. Phase H is launch-day required.

Where Triskope is today

The current crm-proto repository is a polished, single-file React prototype deployed via GitHub Pages. Everything visible in the UI — the leads, the agents, the listings, the AI output, the notifications — is hardcoded into the JavaScript bundle. The app has no database, no authentication, no payment processing, no real form submissions, and no real MLS data. Visitors to thomasbass26-del.github.io/crm-proto see the same demo state, no matter who they are.

This is intentional and appropriate for a prototype. It is fast to iterate on, costs nothing to host, and is perfect for demos. But it is not yet a product anyone can sign up for or pay for.

Where Triskope needs to go

To run as a real SaaS where agents pay monthly and get their own branded site, three layers need to come into being on top of the existing front-end. They can be built in roughly the order they appear in this document.

- A multi-tenant backend (database, authentication, role-based access)
- A billing system (Stripe Checkout + webhook-driven access control)
- A lead-capture pipeline (public forms on community pages that route into the correct agent's CRM)

Around those three core layers sit several supporting integrations: subdomain hosting for agent-branded sites, an admin panel for granting and denying access, real MLS data, real AI calls, transactional email, analytics, and the legal scaffolding required to charge money for software.

TOTAL ESTIMATED EFFORT

Approximately 6-8 weeks of focused build time to reach a state where a real first agent can sign up, pay, capture a lead from their community page, and have it flow into their CRM. Plus ongoing operational cost between $25-1,050 per month depending on agent count.

Phase A — Backend foundation

Estimated time: 1 week. Estimated cost: $25/month (Supabase Pro) once paying customers exist; $0 during dev.

Everything else in this roadmap depends on having a real database with auth and access control. The recommended platform is Supabase: it provides a hosted Postgres database, an authentication system with email/password and social logins, row-level security policies that enforce per-agent data isolation at the database layer, and an auto-generated REST and realtime API. The combination eliminates roughly three weeks of backend boilerplate.

Tasks

- ☐ Create a Supabase project at supabase.com. Start on the free tier.
- ☐ Copy the project URL and the anon/service keys into a secure password manager.
- ☐ Decide on email provider: Supabase's built-in for development, switch to Resend or Postmark for production transactional email.
- ☐ Define the database schema (see table list below) using Supabase's SQL editor or the schema designer.
- ☐ Enable Row Level Security on every table that contains agent-scoped data.
- ☐ Write RLS policies: agents can read or write rows where agent_id = auth.uid(); admins (role='admin') can read everything.
- ☐ Configure authentication providers: email/password is required. Optionally enable Google OAuth for faster signup.
- ☐ Create your own admin user (you) with role='admin' to bootstrap the system.
- ☐ Set the daily backup retention policy to at least 7 days (Supabase Pro extends this to 30 days).
- ☐ Add a staging project mirroring production, so schema changes can be tested before going live.

Database schema (recommended starting tables)

Every table should have created_at and updated_at timestamps. Foreign keys are noted in parentheses.

- users — id, email, role (admin | agent | visitor), display_name. Auth-linked.
- agents — user_id (→ users), full_name, plan, stripe_customer_id, stripe_subscription_id, access_status (active | past_due | suspended | canceled), subdomain, headshot_url, bio.
- leads — id, agent_id (→ agents), name, email, phone, status (new | nurture | hot | cold | closed), score, source, area, budget, interest, ai_notes, stage.
- lead_tags — lead_id (→ leads), tag. Many-to-many.
- lead_activity — id, lead_id (→ leads), type, text, occurred_at. Append-only event log.
- lead_notes — id, lead_id (→ leads), agent_id (→ agents), text.
- lead_tasks — id, lead_id (→ leads), agent_id (→ agents), text, due_date, done.
- communities — id, name, slug, area, type. Platform-level catalog.
- agent_communities — agent_id, community_id. Which communities each agent is subscribed to.

- community_pages — id, agent_id, community_id, custom_content (JSONB). Per-agent instance of a community landing page.
- listings — id, mls_id, address, community_id, price, beds, baths, sqft, type, status, lat, lng, listing_agent, days_on_market. Synced from MLS.
- saved_listings — agent_id, listing_id, saved_at.
- subscriptions — id, agent_id, stripe_id, plan, status, current_period_end. Mirrors Stripe state for fast access checks.
- notifications — id, user_id, type, title, text, lead_id, read_at.
- audit_log — id, actor_user_id, action, target_table, target_id, payload. Tracks admin actions for accountability.

Phase B — Migrate frontend to real data

Estimated time: 1-2 weeks. No new monthly cost.

With the backend in place, the existing prototype needs to be rewired to read and write through Supabase instead of using hardcoded arrays. The good news is the React component structure stays mostly intact — only the data source changes.

Tasks

- ☐ Install @supabase/supabase-js and configure a client with the project URL + anon key (kept in a .env file, never committed).
- ☐ Build a login page (email + password) and a signup page (email + password + plan selection).
- ☐ Gate the entire app behind authentication. Unauthenticated visitors land on a marketing page; authenticated visitors land on the dashboard.
- ☐ Wrap the app in an AuthContext that exposes the current user, their role, and a sign-out function.
- ☐ Replace the hardcoded LEADS array with a Supabase query, scoped automatically by RLS.
- ☐ Replace the hardcoded AGENTS array similarly (admin-only view).
- ☐ Replace the hardcoded LISTINGS array with a Supabase query, eventually backed by MLS sync.
- ☐ Convert lead status / pipeline / notes / tasks mutations from useState updates to Supabase inserts and updates.
- ☐ Subscribe to lead changes via Supabase Realtime so new leads appear in the Inbox live.
- ☐ Add an agent settings page: display name, subdomain (with availability check), bio, headshot upload via Supabase Storage.
- ☐ Implement role-based routing: admins see the admin panel nav; agents see only their own data.
- ☐ Show a clear loading state on each view while the initial fetch resolves.
- ☐ Add error boundaries so a network failure does not produce a black screen.

Phase C — Stripe billing

Estimated time: 1 week. Stripe fees: 2.9% + 30¢ per successful transaction. No monthly platform fee on the standard plan.

Stripe is the standard for SaaS subscription billing and the easiest path to taking real payments. The strategy is to let Stripe own all payment state and have your backend mirror only what is needed for access checks.

Tasks

- ☐ Create a Stripe account at stripe.com. Activate the live keys after business verification.
- ☐ Create three Products in Stripe: Starter, Pro, Enterprise. Each gets a recurring monthly Price ($49, $99, $199).
- ☐ Optionally add annual prices with a discount (e.g., 2 months free) to incentivize yearly commits.
- ☐ Enable the Stripe Customer Portal in the Stripe Dashboard so agents can self-serve plan changes, payment methods, and cancellation.
- ☐ Build the signup flow: agent picks a plan → app calls Stripe Checkout Session API → agent enters card → Stripe redirects back to triskope.io/welcome.
- ☐ Create a Supabase Edge Function as a Stripe webhook handler. Listen for customer.subscription.created, customer.subscription.updated, customer.subscription.deleted, invoice.payment_failed, invoice.payment_succeeded.
- ☐ On each webhook event, update agents.access_status and subscriptions table accordingly. Idempotency key: Stripe's event.id.
- ☐ Surface a 'Manage billing' link in agent settings that opens the Stripe Customer Portal.
- ☐ Add a billing-warning banner that appears at the top of the app when access_status = past_due.
- ☐ When access_status = suspended or canceled, redirect agent to a 'Reactivate' page with the Customer Portal link.
- ☐ Test the full lifecycle with Stripe test cards: signup, upgrade, downgrade, cancel, payment failure, payment recovery.
- ☐ Set up Stripe email receipts and brand them with the Triskope logo.

Access status state machine

Status	What it means	Agent can log in?	Subdomain serves site?
active	Subscription current	Yes	Yes
past_due	Recent payment failed, in retry window	Yes (with warning)	Yes
suspended	Manually paused by admin	Redirected to billing	Shows 'temporarily unavailable'
canceled	Subscription ended (user-initiated)	Redirected to reactivate	Shows reactivate CTA

Phase D — Lead magnet → CRM wiring

Estimated time: 1 week. No new monthly cost beyond what was already in Phase A and Phase C.

This is the core value proposition of the platform: a visitor lands on an agent's community page, fills out a form, and the lead appears in that agent's CRM with an auto-generated AI score. The plumbing is straightforward but the routing logic deserves careful design.

Tasks

- ☐ Build reusable form components: ContactForm, MarketReportRequest, HomeValuationRequest, CommunityInterest.
- ☐ Each form posts to a Supabase Edge Function: ingestLead(payload, agent_id, community_id, source).
- ☐ Validate inputs server-side (email format, phone normalization, anti-spam honeypot field, basic rate limiting by IP).
- ☐ Insert the lead with status='new' and tags derived from the form (e.g., 'pre-approved-buyer' if a checkbox is checked).
- ☐ Lead routing logic: if the form is on an agent's branded subdomain → assign to that agent. If on a shared community page → round-robin across subscribed agents or by territory.
- ☐ Trigger an AI auto-score on intake by calling OpenAI/Claude with the lead profile + session signals. Save the score and ai_notes.
- ☐ Send a transactional email to the assigned agent within 60 seconds (Resend or Postmark).
- ☐ Insert a notification row so the lead also shows in the agent's Inbox immediately.
- ☐ Track UTM parameters and referrer on the lead row for attribution reporting.
- ☐ Add an admin-side reassignment UI in case routing needs manual override.

Two routing models — pick which you want to offer per plan tier

Routing model	How it works	Best for
Per-agent	Lead form on agent's own subdomain (sarah.triskope.io/community/grande-dunes) routes directly to that agent.	Pro and Enterprise tiers — gives the agent full ownership of inbound leads
Round-robin	Shared community page (triskope.io/community/grande-dunes) routes to subscribed agents in rotation.	Starter tier — shared lead pool, cheaper plan
Territory	Shared community page routes by visitor's ZIP / property area to the agent who owns that area.	Enterprise tier — exclusive territory rights

Phase E — Subdomain routing and hosting

Estimated time: 1 week, plus 24-72 hours for DNS propagation. Hosting cost varies by choice.

Triskope's value to an agent partly comes from getting a branded subdomain (sarahmitchell.triskope.io). This requires a hosting setup that supports wildcard subdomains plus an app-level router that detects the subdomain and serves the right agent's site. Hostinger's shared hosting is not ideal for this. The recommendation is to host the app on Vercel or Cloudflare Pages, with the marketing site optionally on Hostinger.

Hosting decision

Host	Wildcard subdomains?	Node/SSR support?	Monthly cost
Hostinger Shared	Limited (manual per-sub)	No (PHP only)	$3-15
Hostinger VPS	Yes (you manage Nginx)	Yes (Node + reverse proxy)	$8-30
Vercel (recommended)	Yes (native)	Yes (Next.js, Edge functions)	Free or $20
Cloudflare Pages	Yes (native)	Yes (Workers)	Free or $5

Tasks

- ☐ Decide which host will run the app. Vercel is the recommended path.
- ☐ If keeping Hostinger for triskope.io (marketing only): point the root domain to Hostinger via A record.
- ☐ Point *.triskope.io (wildcard) to the chosen app host via CNAME or A record. Vercel and Cloudflare provide wildcard DNS instructions.
- ☐ In Vercel/Cloudflare, add triskope.io and *.triskope.io as custom domains on the app project.
- ☐ Issue wildcard SSL certificate (handled automatically on Vercel and Cloudflare; manual on Hostinger VPS).
- ☐ Implement subdomain detection in the app: parse window.location.hostname → extract subdomain → look up agent by subdomain in DB.
- ☐ If agent exists and access_status=active, render their branded site. Otherwise redirect to triskope.io with a 'site not found' message.
- ☐ During signup, validate subdomain availability and enforce a reserved list (admin, app, www, api, mail, etc.).
- ☐ Build the public marketing site at triskope.io: home page, pricing, features, signup CTA, demo video, footer.

Note on Hostinger

Hostinger shared plans are excellent for static marketing sites and WordPress, but they do not run Node.js apps well and only handle wildcard subdomains on VPS plans. If you are committed to

Hostinger, plan to use a VPS (~$8-30/month) and configure Nginx as a reverse proxy yourself, or use the shared plan only for triskope.io's marketing pages while the actual app runs on Vercel.

Phase F — Admin panel

Estimated time: 1 week. No new monthly cost.

This is the panel you use as the platform owner to manage subscribers, grant access, deny access, refund, and see how the business is performing. It lives inside the same React app but is only visible to users with role='admin'.

Tasks

- ☐ Add an Admin section in the sidebar nav, conditionally rendered when user.role === 'admin'.
- ☐ Build an All Agents view: table of every agent showing name, plan, MRR contribution, lead count, last login, access status, signup date.
- ☐ Add per-agent detail with action buttons: Suspend, Resume, Change Plan, Comp Free Month, Refund Last Payment, Reset Password, Reassign Leads, Delete Account.
- ☐ Every admin action writes to audit_log (who did what when, on whom).
- ☐ MRR dashboard: current MRR, last-month MRR, growth rate, churn rate, ARPU, agent count by plan, projected annual run rate.
- ☐ Failed payments view: list of agents in past_due, days overdue, retry attempts remaining.
- ☐ System health: error rates, API latency, daily signups, daily active agents, daily leads captured.
- ☐ Bulk operations: send a system-wide announcement email, mass-comp a month for a referral period, export data to CSV.
- ☐ Add an Impersonate button that lets you log in as an agent (for support troubleshooting). Audit-log every impersonation.

Phase G — Production integrations

Each integration can be added independently after the core platform is live. Some are critical for the product to feel real (MLS, AI); others can be deferred (SMS, advanced analytics).

MLS data feed

To replace the mock listings with real Grand Strand inventory, an MLS data feed is required. Options listed in order of recommendation.

- Spark API by FBS Data Systems — $100-200/month — well documented, RESO Web API standard, fast to integrate.
- Bridge Interactive — pricing varies, deeper data set, more setup work.
- Trestle by CoreLogic — broad coverage, requires MLS approval per board.

☐ Apply for MLS access through Coastal Carolinas Association of Realtors (the local MLS board).

☐ Get approval from the MLS for data access and API credentials.

☐ Subscribe to Spark API or chosen feed.

☐ Build a daily (or 15-min) sync job: fetch new/updated listings → upsert into listings table → emit listing_added events.

☐ Display correct MLS attribution and disclaimers on listing pages (compliance requirement).

Real AI calls

Currently the AI panel uses hardcoded templates with randomized variations. To replace these with real AI:

☐ Choose OpenAI (gpt-4o or gpt-4o-mini) or Anthropic (Claude Sonnet or Haiku). Both work well; pricing similar.

☐ Move AI calls server-side via Supabase Edge Functions so API keys never leak to the browser.

☐ Add per-agent rate limiting (e.g., 25 generations/day on Starter, 100 on Pro, unlimited on Enterprise).

☐ Implement streaming responses for that real-time typewriter feel (already simulated in the UI).

☐ Cache repeated generations (e.g., market reports) to reduce cost.

Email

Two flavors of email: transactional (signup confirmations, lead notifications, password resets) and marketing (drip campaigns, newsletters).

☐ Sign up for Resend (recommended, $20/month for 50K transactional emails) or Postmark.

☐ Configure SPF, DKIM, and DMARC records on triskope.io for deliverability.

☐ Design and code transactional templates: welcome, new lead alert, payment failed, weekly digest, lead unread reminder.

☐ Optional: integrate with Mailchimp or ConvertKit for marketing sequences if you want a polished marketing automation tool.

SMS (optional but high-value for hot leads)

- ☐ Sign up for Twilio. SMS costs ~$0.0075 per message in the US.
- ☐ Add agent setting: phone number for SMS alerts. Verify via one-time code.
- ☐ Send SMS alert when a hot lead arrives (score ≥ 85) or when an in-app task is overdue.

Analytics

- ☐ Add Plausible ($9/month, privacy-friendly) or PostHog (free tier, more features) tracking script to public pages.
- ☐ Track signups, lead submissions, agent engagement, churn signals (last login age, plan downgrades).
- ☐ Build a /metrics admin page that surfaces high-signal numbers without needing to open the analytics dashboard.

Phase H — Legal, compliance, and launch checklist

These items are non-optional before charging real money for software. The legal pieces require either a lawyer or a vetted DIY service like TermsFeed; do not skip them.

Legal documents

☐ Terms of Service — describes the service, user obligations, limitation of liability, governing law. Lawyer-reviewed strongly recommended.

☐ Privacy Policy — what data is collected, how it is used, retention, third parties (Stripe, Supabase, Resend, etc.), user rights. Required by GDPR if you have any EU visitors and by CCPA for California.

☐ Cookie banner if any tracking cookies are used. Plausible is cookie-free; PostHog and Google Analytics are not.

☐ Acceptable Use Policy — what agents can and cannot post via the platform (no harassment, spam, false listings, etc.).

☐ Subscription Agreement / Master Services Agreement — embedded in the Stripe Checkout signup flow with explicit acceptance checkbox.

☐ DMCA notice and takedown procedure — required for any user-generated content.

☐ Real estate compliance: state-specific disclosures for IDX listings (Fair Housing, Equal Opportunity, brokerage attribution). The MLS provider will give specifics.

Insurance

☐ Errors & Omissions / Professional Liability — protects you if a software bug or AI suggestion contributes to an agent's bad outcome. Typical cost $500-1,500/year.

☐ Cyber Liability — covers data breaches and ransomware. Typical cost $1,000-3,000/year.

☐ General Liability if you ever host an event or take in-person meetings.

Operational readiness

☐ Support email (support@triskope.io) routed to a real inbox you actively monitor.

☐ Help center: Notion or Intercom Articles — minimum content is FAQ, getting started, how-to-add-a-community-page, how-to-cancel.

☐ Status page (StatusPage.io has a free plan, or Better Uptime) so agents know if there's an outage.

☐ Daily database backups verified by restoring to staging at least once.

☐ Error monitoring via Sentry (free tier handles small scale).

☐ Uptime monitoring via Better Uptime or UptimeRobot.

☐ Incident response runbook: who you call, what you check, how you communicate with agents during an outage.

Pre-launch marketing

☐ Polished landing page at triskope.io with hero, features, pricing, demo video, testimonials (placeholder until you have them), and a clear sign-up CTA.

☐ Demo video (3-5 min) recorded with Loom: walk through Dashboard → Pipeline → Lead detail → AI tools → Listings → Inbox.

☐ Pricing page with side-by-side plan comparison, FAQ, and money-back guarantee terms.

☐ Onboarding email sequence (Day 0 welcome, Day 1 setup checklist, Day 3 product tour, Day 7 office-hours invite).

☐ Curated outreach list of 10-20 Grand Strand agents you personally know. Offer them a comped first three months in exchange for feedback.

☐ Soft launch with the first 5 agents. Watch every signup. Talk to every agent weekly.

☐ Once five agents are happily using the product and paying, open public signups.

Estimated monthly costs at scale

Costs grow with agent count and feature usage. Numbers below are reasonable ballparks for planning. Stripe fees scale linearly with revenue (2.9% + 30¢ per transaction).

Service	At 0 agents (dev)	At 10 agents	At 50 agents	At 100 agents
Supabase	Free	$25	$25	$25-100
Vercel hosting	Free	$20	$20	$20
Stripe fees (2.9%+30¢)	$0	~$30	~$150	~$300
MLS API (Spark)	$0	$150	$150	$200
OpenAI / Claude API	$0	$30	$200	$700
Resend email	Free	$20	$20	$50
Plausible analytics	Free	$9	$9	$19
Sentry monitoring	Free	Free	$26	$26
Better Uptime	Free	Free	$24	$24
Domain + email forwarding	$1.50	$1.50	$1.50	$1.50
Total infrastructure	~$2	~$285	~$625	~$1,365

Revenue at the same scale

Assumes a typical mix of plans: 30% Starter, 60% Pro, 10% Enterprise. Average revenue per agent (ARPU) ≈ $99/month.

Agents	Gross MRR	Stripe fees	Infrastructure	Net MRR
10	$990	~$30	~$255	~$705
50	$4,950	~$150	~$475	~$4,325
100	$9,900	~$300	~$1,065	~$8,535
250	$24,750	~$750	~$2,800	~$21,200

Recommended vendors and quick reference

Save this page; these are the providers worth signing up with and the specific pages to start on.

Core infrastructure

- Supabase — supabase.com — database, auth, storage, edge functions
- Vercel — vercel.com — frontend hosting with wildcard subdomain support
- Stripe — stripe.com — subscription billing, customer portal, webhooks
- Resend — resend.com — transactional email with React-style templates
- Cloudflare — cloudflare.com — DNS, CDN, optional WAF (free tier is generous)

Real estate specific

- Spark Platform — sparkplatform.com — MLS data feed via RESO Web API
- Coastal Carolinas Association of Realtors — local MLS board for Grand Strand
- Bridge Interactive — bridgedataoutput.com — alternative MLS data
- Trestle — trestle.corelogic.com — alternative MLS aggregator

AI

- OpenAI Platform — platform.openai.com — GPT-4o and GPT-4o-mini
- Anthropic — anthropic.com/api — Claude Sonnet and Haiku

Operations and compliance

- Sentry — sentry.io — error monitoring
- Better Uptime — betteruptime.com — uptime monitoring and incident response
- Plausible — plausible.io — privacy-friendly analytics
- Notion — notion.so — help center and runbooks
- TermsFeed — termsfeed.com — DIY legal documents if not hiring a lawyer

Domain

- triskope.io — already registered. Confirm WHOIS privacy is enabled.
- Configure DNS at your registrar to point root + wildcard to your chosen host.

NEXT STEP SUGGESTION

When you're ready to start the real build, the highest-leverage first task is creating the Supabase project and locking in the database schema. Everything downstream depends on it. Block one focused half-day for it and you'll have a working backend foundation before lunch.